

Guía del alumno

Módulo 2, Manipulación de datos

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía es para leer después de la clase. Trae el mismo código y las mismas cifras que viste en el laboratorio, con la explicación escrita, así que no hace falta tomar apuntes de código durante la sesión.

Cada bloque tiene la misma estructura. Primero qué se vio y para qué sirve. Después el código celda por celda con su explicación. Al final una tabla de referencia rápida, los errores más comunes con el mensaje tal como aparece en pantalla, la solución del ejercicio y tres preguntas para comprobar que quedó claro.

La última página es una hoja de referencia con todo el módulo resumido, pensada para imprimir.

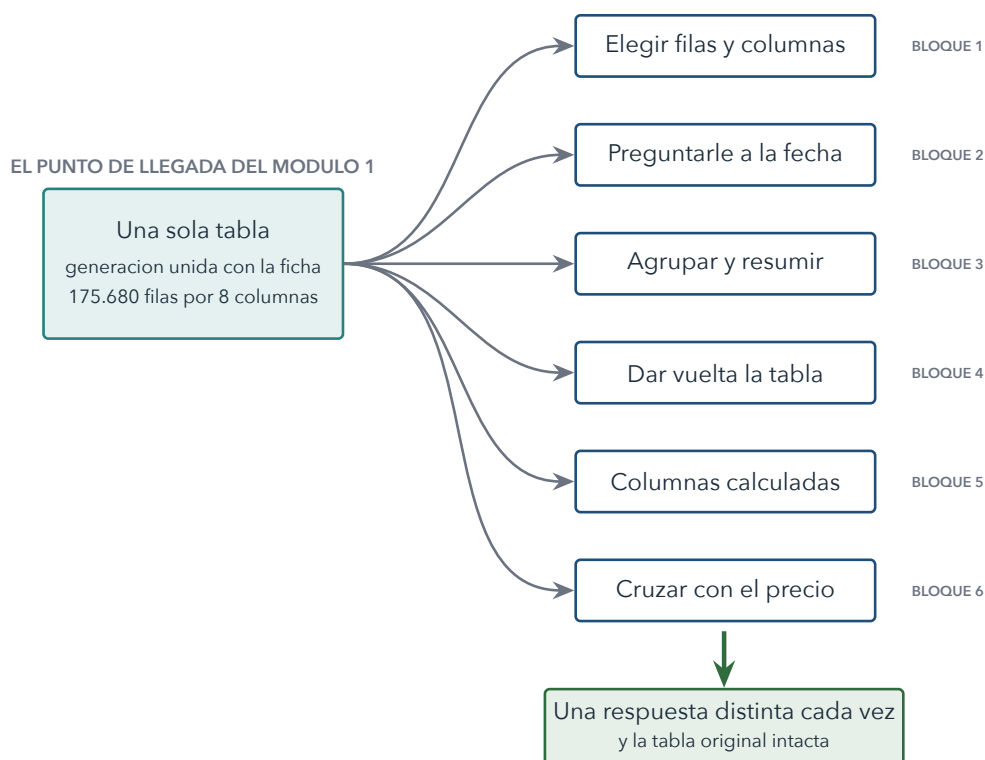
Lo que vas a poder hacer al terminar

- Elegir filas y columnas de una tabla grande escribiendo la condición, no marcándolas a mano.
- Pedirle a una fecha el mes, el día de la semana, la hora o el trimestre.
- Agrupar por una o varias claves y calcular varias cosas de una vez.
- Dar vuelta una tabla para que alguien la pueda leer, y deshacerla para volver a calcular.
- Crear columnas que no venían en el archivo, clasificar con cortes y repartir un total entre sus filas.
- Cruzar dos fuentes que se identifican por más de una columna.

Todas las cifras de esta guía salen de la corrida verificada del notebook con pandas 2.3.3. Los datos son los mismos del Módulo 1, sintéticos y generados dentro del notebook. Ninguna central corresponde a una instalación real, y donde una cifra del laboratorio se aleja de la realidad del sistema chileno, la guía lo dice.

1. El módulo en una imagen

Todo el módulo pasa sobre una sola tabla, la que quedó unida al final del Módulo 1. Lo que cambia en cada bloque es la pregunta, no los datos.



La imagen del patio

Seguimos en el puerto del módulo anterior, pero ya no en el muelle. Ahora estamos en el patio de operaciones.

La carga está bajada, está en pallets iguales y cada pallet tiene su documentación al lado. Llega el jefe de turno y pregunta cuánto se movió ayer de carga refrigerada. Llega el de facturación y pregunta cuánto salió por cliente. Llega la autoridad y pide el total del mes por tipo de carga.

Ninguna de esas preguntas mueve un solo pallet. El patio responde marcando con tiza, contando por montón y anotando en una planilla. La carga sigue donde estaba.

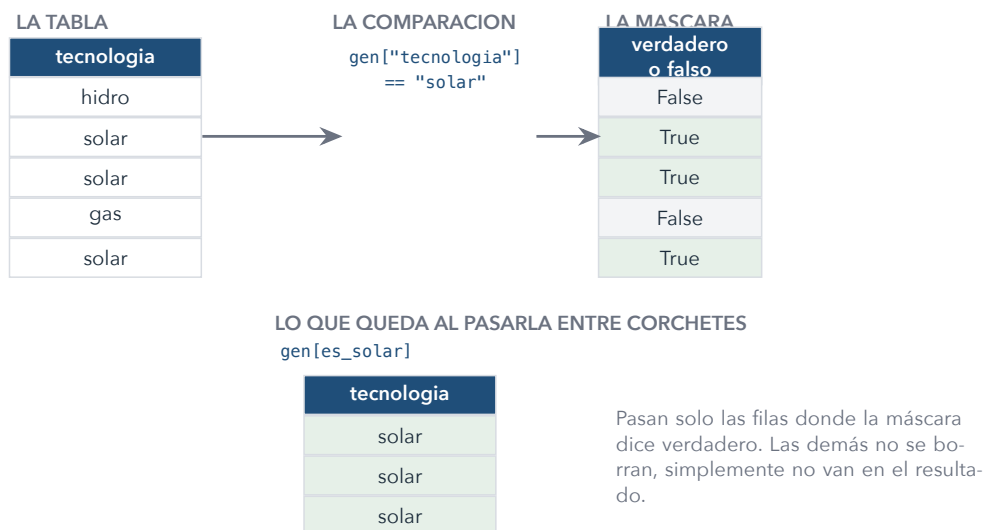
Eso es este módulo. La tabla no se toca. Lo que se produce es una tabla nueva, más chica, resumida o dada vuelta, según quién haya preguntado.

2. Bloque 1, elegir lo que importa

Qué se vio

Filtrar es marcar con tiza, no mover carga. Cuando eliges un pedazo de la tabla, la tabla original queda igual y lo que obtienes es una tabla nueva. Si no la guardas en un nombre, se pierde.

Hay dos cosas que se eligen, columnas y filas. Las columnas se eligen pasando una lista de nombres. Las filas se eligen pasando una columna de verdaderos y falsos, que se llama máscara y es el concepto central del bloque.



El código, paso a paso

1.1 El punto de partida

```
import pandas as pd

# El punto de partida de este lab es el punto de llegada del anterior.
generacion = pd.read_csv("generacion.csv", parse_dates=["fecha"])
centrales = pd.read_csv("centrales.csv")
demanda = pd.read_csv("demanda.csv", parse_dates=["fecha"])

# La tabla unida del Lab 01, generación con la ficha de cada central pegada al lado.
gen = pd.merge(generacion, centrales, on="central", how="left")

print("gen      ", gen.shape, list(gen.columns))
print("demanda", demanda.shape, list(demanda.columns))
```

```
gen      (175680, 8) ['fecha', 'hora', 'central', 'mwh', 'tecnologia', 'region', 'potencia_mw', 'anio_inicio']
demanda (61488, 4) ['fecha', 'hora', 'region', 'mwh']
```

La tabla gen es la del Módulo 1, la generación horaria con la ficha de cada central pegada al lado. Tiene 175.680 filas y 8 columnas. La tabla demanda viene aparte y se usa recién en el bloque 6.

1.2 Elegir columnas

```
# Los corchetes de afuera son "dame esto", los de adentro son la lista de columnas.
gen[["fecha", "hora", "central", "tecnologia", "mwh"]].head()
```

	fecha	hora	central	tecnologia	mwh
0	2024-01-01	0	Central Rio Manso Alto	hidro	163.69
1	2024-01-01	1	Central Rio Manso Alto	hidro	179.81
2	2024-01-01	2	Central Rio Manso Alto	hidro	173.77
3	2024-01-01	3	Central Rio Manso Alto	hidro	170.39
4	2024-01-01	4	Central Rio Manso Alto	hidro	169.84

Los corchetes de afuera son la selección y los de adentro son la lista de columnas. Por eso se ven dos corchetes seguidos. Si pides una sola columna sin lista, `gen["mwh"]`, obtienes una Serie, que es una columna suelta. Con lista, `gen[["mwh"]]`, obtienes una tabla de una columna.

1.3 loc, por nombre

```
# loc trabaja con nombres. Antes de la coma van las filas, después las columnas.
# Ojo, loc incluye las dos puntas, así que 0:4 son cinco filas.
gen.loc[0:4, ["central", "tecnologia", "mwh"]]
```

	central	tecnologia	mwh
0	Central Rio Manso Alto	hidro	163.69
1	Central Rio Manso Alto	hidro	179.81
2	Central Rio Manso Alto	hidro	173.77
3	Central Rio Manso Alto	hidro	170.39
4	Central Rio Manso Alto	hidro	169.84

`loc` trabaja con nombres. Antes de la coma van las filas y después las columnas. Incluye las dos puntas, así que `0:4` devuelve cinco filas.

1.4 iloc, por posición

```
# iloc trabaja con posiciones y deja fuera la última, así que 0:4 son cuatro filas.
# Acá además pedimos las columnas 2, 5 y 6, en ese orden.
gen.iloc[0:4, [2, 5, 6]]
```

	central	region	potencia_mw
0	Central Rio Manso Alto	Biobio	420
1	Central Rio Manso Alto	Biobio	420
2	Central Rio Manso Alto	Biobio	420
3	Central Rio Manso Alto	Biobio	420

`iloc` trabaja con posiciones y deja fuera la última, así que `0:4` devuelve cuatro filas. Como las columnas van por número, si cambia el orden del archivo la misma línea entrega otra cosa sin avisar. Por eso en el trabajo real se prefiere `loc`.

1.5 La máscara

```
# Una comparación sobre una columna no devuelve un dato, devuelve una columna
# de verdaderos y falsos, una por fila.
es_solar = gen["tecnologia"] == "solar"

print(es_solar.head(3))
print(es_solar.sum(), "filas cumplen la condición, de", len(gen))
```

```
0    False
1    False
2    False
Name: tecnologia, dtype: bool
43920 filas cumplen la condición, de 175680
```

Una comparación sobre una columna no devuelve las filas que cumplen, devuelve una columna de verdaderos y falsos, una por cada fila de la tabla. Eso es la máscara. Las 43.920 filas que cumplen son cinco centrales solares por las 8.784 horas del año 2024.

1.6 La máscara entre corchetes

```
# Esa columna de verdaderos y falsos es la que se pasa entre corchetes.
solares = gen[es_solar]
print(solares.shape)

# Lo mismo escrito de corrido, que es como se ve en el código real.
print(gen[gen["tecnologia"] == "solar"].shape)
```

```
(43920, 8)
(43920, 8)
```

Recién al pasar la máscara entre corchetes quedan las filas. Las dos formas, la de dos pasos y la de corrido, dan lo mismo, y en el código real vas a ver la de corrido.

1.7 Varias condiciones

```
# & es "y", | es "o", ~ es "no". Cada condición va entre paréntesis.
solar_norte = gen[(gen["tecnologia"] == "solar") & (gen["region"] == "Antofagasta")]
print("solar en Antofagasta      ", solar_norte.shape[0], "filas")

no_termicas = gen[~gen["tecnologia"].isin(["gas", "carbon", "diesel"])]
print("todo lo que no es térmico ", no_termicas.shape[0], "filas")
```

```
solar en Antofagasta      26352 filas
todo lo que no es térmico 122976 filas
```

& es y, | es o, y ~ es no. Cada condición va entre paréntesis, sin excepción, porque en Python esos operadores se evalúan antes que la comparación. No se usan las palabras and y or, porque esas comparan un valor contra otro y acá estamos comparando columnas completas.

1.8 isin y between

```
# isin pregunta si el valor está en una lista y reemplaza a varios | seguidos.
sur = gen[gen["region"].isin(["Biobio", "Los Lagos"])]
print("regiones del sur", sur["region"].unique())

# between pregunta por un rango, con los dos extremos incluidos.
grandes = centrales[centrales["potencia_mw"].between(200, 400)]
print("centrales entre 200 y 400 MW", grandes.shape[0])
```

```
regiones del sur ['Biobio' 'Los Lagos']
centrales entre 200 y 400 MW 9
```

isin pregunta si el valor está en una lista y reemplaza a varios | seguidos. between pregunta por un rango con las dos puntas incluidas. Fíjate en que la pregunta por potencia se le hace a centrales y no a gen, porque la potencia es un dato de la central y no de cada hora.

1.9 query

```
# query recibe la condición escrita como texto, sin corchetes ni paréntesis.
print(gen.query("tecnologia == 'solar' and region == 'Antofagasta').shape)

# El mismo resultado que el filtro de más arriba, escrito de otra forma.
```

```
(26352, 8)
```

query recibe la condición como texto y entrega el mismo resultado. Es cómodo para condiciones simples y para quien viene de SQL. Se pone incómodo cuando la condición se arma con variables.

La solución del ejercicio

El ejercicio pedía cambiar la pregunta a generación eólica o hidro, en Biobio o Los Lagos, entre el 1 de junio y el 31 de agosto.

```
respuesta = gen[
    gen["tecnologia"].isin(["eolica", "hidro"])
    & gen["region"].isin(["Biobio", "Los Lagos"])
    & gen["fecha"].between("2024-06-01", "2024-08-31")
]

print(respuesta.shape[0], "filas")
print(round(respuesta["mwh"].sum(), 1), "MWh")
```

El resultado son 11.040 filas y 1.882.237,0 MWh. Las 11.040 filas son 5 centrales por 92 días por 24 horas. Ese cálculo de comprobación conviene hacerlo siempre, porque si el número no calza es que el filtro no dice lo que creías.

Referencia rápida

Instrucción	Para qué sirve
<code>df[["a", "b"]]</code>	Elegir columnas por su nombre
<code>df.loc[filas, columnas]</code>	Elegir por nombre, incluye las dos puntas
<code>df.iloc[filas, columnas]</code>	Elegir por posición, deja fuera la última
<code>df["col"] == valor</code>	Construye la máscara de verdaderos y falsos
<code>df[mascara]</code>	Deja solo las filas donde la máscara es verdadera
<code>&, , ~</code>	Y, o, no. Cada condición entre paréntesis
<code>df["col"].isin([...])</code>	El valor está en la lista
<code>df["col"].between(a, b)</code>	El valor está en el rango, con las puntas incluidas
<code>df.query("a == 1 and b > 2")</code>	La condición escrita como texto
<code>df["col"].str.contains("x")</code>	Filtrar por texto parcial

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>TypeError: unsupported operand type(s) for &</code>	Faltan paréntesis alrededor de cada condición. Es el error más frecuente del bloque y el mensaje no habla de paréntesis.
<code>ValueError: The truth value of a Series is ambiguous</code>	Usaste <code>and</code> o <code>or</code> en vez de <code>&</code> o <code> </code> .
<code>KeyError: 'Region'</code>	El nombre de la columna no es exactamente ese. Revisa con <code>df.columns</code> .
El resultado tiene cero filas y no hay error	La condición no la cumple ninguna fila. Casi siempre es un valor mal escrito, por ejemplo "Solar" con mayúscula cuando el dato dice "solar".

Tres preguntas de repaso

1. Qué devuelve exactamente `gen["tecnologia"] == "solar"`

Una columna de verdaderos y falsos, con una fila por cada fila de la tabla. No devuelve las filas solares.

2. Por qué `loc` y `iloc` no dan el mismo número de filas con `0:4`

Porque `loc` incluye las dos puntas y devuelve cinco filas, mientras `iloc` deja fuera la última y devuelve cuatro.

3. Si filtras y no guardas el resultado, qué pasa con la tabla original

Nada. Filtrar nunca modifica la tabla original. El resultado se imprime y se pierde si no lo guardas en un nombre.

3. Bloque 2, las fechas como herramienta

Qué se vio

El reloj del patio. Cada dato del sector viene con su hora exacta, y de esa hora completa se deducen después todas las preguntas que nadie hizo en el momento. El mes, el día de la semana, el trimestre. La condición para que eso funcione es que la fecha esté leída como fecha y no como texto, que es lo que se resolvió en el Módulo 1 con `parse_dates`. Con la fecha bien leída, el accesorio `dt` entrega todo lo demás.

Dos periodos solo se comparan si duran lo mismo. Es la trampa más común del trabajo con fechas y no tiene nada que ver con pandas. Febrero tiene veintiocho días y marzo treinta y uno, así que comparar sus totales le regala a marzo un diez por ciento. Un año bisiesto contra uno que no lo es le regala un cero coma tres.

Lo que se compara es el **promedio diario**, o sea el total dividido por el número de días de cada periodo. Suena obvio escrito así, y el error de saltárselo es del mismo tamaño que las variaciones que uno anda buscando, que es lo que lo hace peligroso. Si comparas dos periodos, lo primero que hay que preguntarse es cuántos días tiene cada uno.

El código, paso a paso

2.1 Lo que se le puede pedir a una fecha

```
# Todo esto sale de la misma columna fecha, sin cortar texto a mano.
print("año      ", demanda["fecha"].dt.year.head(2).tolist())
print("mes      ", demanda["fecha"].dt.month.head(2).tolist())
print("día      ", demanda["fecha"].dt.day_name().head(2).tolist())
print("día número ", demanda["fecha"].dt.dayofweek.head(2).tolist())
```

```
año      [2024, 2024]
mes       [1, 1]
día       ['Monday', 'Monday']
día número [0, 0]
```

El año, el mes, el nombre del día y el día como número, todo desde la misma columna. En `dayofweek` el cero es lunes y el seis es domingo. El primero de enero de 2024 fue lunes, por eso aparece Monday y 0.

2.2 Las columnas derivadas

```
# Cuatro columnas nuevas, calculadas para las 61.488 filas de una sola vez.
demanda["mes"] = demanda["fecha"].dt.month
demanda["nombre_mes"] = demanda["fecha"].dt.month_name()
demanda["dia_semana"] = demanda["fecha"].dt.dayofweek
demanda["es_finde"] = demanda["dia_semana"] >= 5

demanda.head(3)
```

	fecha	hora	region	mwh	mes	nombre_mes	dia_semana	es_finde
0	2024-01-01	0	Antofagasta	51.95	1	January	0	False
1	2024-01-01	1	Antofagasta	49.02	1	January	0	False
2	2024-01-01	2	Antofagasta	48.22	1	January	0	False

Cuatro columnas nuevas calculadas de una vez para las 61.488 filas. No hay ningún ciclo, se escribe como si fuera un solo valor y pandas lo aplica a toda la columna. Conviene crear las columnas que vas a usar varias veces, no todas las que se podrían crear.

2.3 Lo mismo en la otra tabla

```
# Las dos que más vamos a usar, ahora en la tabla de generación.
gen["mes"] = gen["fecha"].dt.month
gen["es_finde"] = gen["fecha"].dt.dayofweek >= 5

print(gen["es_finde"].value_counts())
```

```
es_finde
False    125760
True      49920
Name: count, dtype: int64
```

`es_finde` es una columna de verdadero y falso, la misma clase de columna que la máscara del bloque anterior, solo que guardada para usarla varias veces. Las 49.920 filas de fin de semana son 104 días por 24 horas por 20 centrales.

2.4 Día hábil contra fin de semana

```
# Demanda promedio por hora, separando día hábil de fin de semana.
print(demanda.groupby("es_finde")["mwh"].mean().round(1))

caida = demanda.groupby("es_finde")["mwh"].mean()
print("el fin de semana consume un", round(100 * (1 - caida[True] / caida[False]), 1),
      "por ciento menos")
```

```
es_finde
False    231.2
True     213.1
Name: mwh, dtype: float64
el fin de semana consume un 7.8 por ciento menos
```

Agrupar por una columna de verdadero y falso devuelve exactamente dos filas. En estos datos la demanda cae 7,8 por ciento el fin de semana.

OJO CON ESTA CIFRA

El 7,8 por ciento es de los datos sintéticos del curso. En el sistema eléctrico real la caída de fin de semana existe y es conocida, pero no hay publicada una cifra oficial de cuánto es. El Coordinador modela la demanda con cuatro días tipo, lunes, de martes a viernes, sábado y domingo, lo que confirma que la diferencia importa, sin dar un número.

2.5 Un rango de fechas

```
# Una semana completa, comparando la columna de fechas contra texto.
semana = demanda[demanda["fecha"].between("2024-07-01", "2024-07-07")]

print("del", semana["fecha"].min().date(), "al", semana["fecha"].max().date())
print(semana.shape[0], "filas, que son 7 días por 24 horas por 7 regiones")
```

```
del 2024-07-01 al 2024-07-07
1176 filas, que son 7 días por 24 horas por 7 regiones
```

Comparar la columna de fechas contra texto funciona porque pandas entiende el texto como fecha. Las 1.176 filas son 7 días por 24 horas por 7 regiones. Ese cálculo de comprobación es la mejor costumbre que te puedes llevar de este bloque.

La solución del ejercicio

El ejercicio pedía crear la columna trimestre y filtrar el tercero.

```
demanda["trimestre"] = demanda["fecha"].dt.quarter

seleccion = demanda[demanda["trimestre"] == 3]
print(seleccion.shape[0], "filas")
```

Son 15.456 filas, que es 92 días de julio, agosto y septiembre por 24 horas por 7 regiones.

Referencia rápida

Instrucción	Para qué sirve
<code>df["fecha"].dt.year</code>	El año
<code>df["fecha"].dt.month</code>	El mes como número
<code>df["fecha"].dt.month_name()</code>	El nombre del mes
<code>df["fecha"].dt.day_name()</code>	El nombre del día
<code>df["fecha"].dt.dayofweek</code>	El día como número, 0 es lunes
<code>df["fecha"].dt.quarter</code>	El trimestre, de 1 a 4
<code>df["fecha"].dt.hour</code>	La hora, cuando viene dentro de la fecha
<code>df["fecha"].between(a, b)</code>	Filtrar un rango de fechas
<code>pd.to_datetime(col)</code>	Convertir a fecha una columna que llegó como texto

Errores comunes

Lo que aparece en pantalla	Qué pasó
AttributeError: Can only use .dt accessor with datetimelike values	La columna no es de tipo fecha. Léela con <code>parse_dates</code> o conviértela con <code>pd.to_datetime</code> .
Las fechas se ordenan mal, el 10 de enero antes que el 2	La columna quedó como texto. Es el mismo problema con otra cara.
El filtro por rango devuelve cero filas	Revisa el formato del texto. Con "2024-07-01" funciona, con "01-07-2024" puede interpretarse al revés.

Tres preguntas de repaso

1. Qué número devuelve `dayofweek` para un domingo

Seis. El cero es lunes.

2. Por qué conviene guardar el mes en una columna en vez de calcularlo cada vez

Porque lo vas a usar varias veces, para filtrar, para agrupar y para pivotar. Si lo usas una sola vez, no hace falta la columna.

3. Qué tienen en común `es_finde` y la máscara del bloque 1

Las dos son columnas de verdadero y falso. Sirven igual para filtrar que para agrupar.

HASTA ACÁ LLEGÓ LA PRIMERA CLASE

Este módulo se dicta en dos sesiones. Los bloques 1 y 2 son de la primera y el resto va en la siguiente.

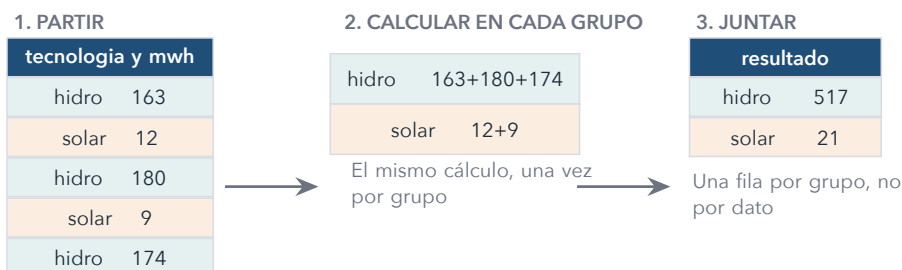
Cuando vuelvas a abrir el notebook, ejecútalo desde arriba con Entorno de ejecución, Ejecutar todo. Colab no guarda el estado entre sesiones, así que aunque veas el código igual que como lo dejaste, las tablas y las columnas ya no están cargadas.

4. Bloque 3, agrupar y resumir en serio

Qué se vio

Contar por montón. Es la operación que más vas a usar en tu trabajo, porque ningún informe se publica por central y por hora, se publica por tecnología, por región o por mes.

groupby tiene tres tiempos. Parte la tabla en grupos según una o varias columnas, aplica el mismo cálculo a cada grupo, y junta los resultados en una tabla nueva que tiene una fila por grupo, no una por dato.



Un total es un promedio ponderado, y lo que promedia lo esconde. Esta es la razón de fondo por la que agrupar existe, y conviene tenerla clara antes que la sintaxis.

Si el sistema completo crece dos por ciento, eso no quiere decir que cada región creció dos por ciento. Puede haber una que creció diez y otra que cayó, y el total no lo dice, porque la región más grande lo arrastra hacia su propio valor. Un informe que entrega solo el total no está mal calculado, está incompleto, y quien tiene que decidir dónde invertir no lo puede usar.

Agrupar no es un paso técnico. Es la diferencia entre un dato y un dato que sirve.

El código, paso a paso

3.1 El caso más simple

```
# Partir por tecnología, sumar mwh en cada grupo, juntar los seis resultados.
gen.groupby("tecnología")["mwh"].sum().round(0).sort_values(ascending=False)
```

```
tecnología
hidro      6106351.0
gas        5094126.0
carbon     4666823.0
eolica     1565899.0
solar      671677.0
diesel     40710.0
Name: mwh, dtype: float64
```

175.680 filas convertidas en seis. En estos datos la hidro encabeza y el diésel casi no aporta. En el sistema chileno real la primera es la solar, porque este laboratorio tiene solo veinte centrales y no pretende reproducir la matriz nacional.

3.2 Dos claves

```
# Con dos claves, el resultado tiene una fila por combinación que exista.
por_region = gen.groupby(["region", "tecnologia"])["mwh"].sum().round(0)

print(por_region.shape[0], "combinaciones de región y tecnología")
por_region.head(6)
```

```
12 combinaciones de región y tecnología
region      tecnologia
Antofagasta carbon      1967910.0
            solar       479681.0
Atacama     solar       191996.0
Biobio      carbon     2698913.0
            gas       1688455.0
            hidro     4151930.0
Name: mwh, dtype: float64
```

Siete regiones por seis tecnologías serían cuarenta y dos combinaciones. Salen doce, porque groupby devuelve solo las combinaciones que existen en los datos. El resultado queda con dos niveles en el índice, y por eso la región no se repite en cada fila.

3.3 Varias funciones a la vez

```
# Con una lista de funciones se calculan todas sobre la misma columna.
gen.groupby("tecnologia")["mwh"].agg(["sum", "mean", "std", "max"]).round(1)
```

```
tecnologia      sum    mean  std   max
carbon      4666823.0  265.6  42.0  318.2
diesel       40709.5    4.6   9.1   24.7
eolica     1565899.4   44.6  26.2  115.2
gas        5094126.1  193.3  45.5  267.7
hidro      6106351.4  139.0  67.3  317.5
solar       671677.2   15.3  21.6  118.2
```

Con una lista de funciones se calculan todas sobre la misma columna. Compara la desviación del carbón, 42,0 sobre un promedio de 265,6, con la de la solar, 21,6 sobre un promedio de 15,3. La solar tiene una desviación más grande que su propio promedio porque de noche genera cero.

3.4 Un cálculo distinto por columna

```
# Con un diccionario, cada columna recibe el cálculo que le corresponde.
gen.groupby("tecnologia").agg({"mwh": "sum", "potencia_mw": "mean",
                             "central": "nunique"}).round(1)
```

	mwh	potencia_mw	central
tecnologia			
carbon	4666823.0	415.0	2
diesel	40709.5	45.0	1
eolica	1565899.4	141.2	4
gas	5094126.1	291.7	3
hidro	6106351.4	253.0	5
solar	671677.2	192.0	5

Con un diccionario, cada columna recibe el cálculo que le corresponde. Sumar los megawatt hora tiene sentido, sumar la potencia instalada no, porque se repetiría en cada hora. Por eso la potencia va con promedio y la central con nunique, que cuenta valores distintos.

3.5 Agregación con nombre

```
# A la izquierda el nombre que quieres, a la derecha la columna y el cálculo.
resumen = gen.groupby(["region", "tecnologia"]).agg(
    total=("mwh", "sum"),
    promedio=("mwh", "mean"),
    centrales=("central", "nunique"),
)

resumen.round(1).head()
```

		total	promedio	centrales
region	tecnologia			
Antofagasta	carbon	1967910.4	224.0	1
	solar	479680.9	18.2	3
Atacama	solar	191996.3	10.9	2
Biobio	carbon	2698912.7	307.3	1
	gas	1688454.7	192.2	1

Esta es la forma que se usa en el trabajo real. A la izquierda el nombre que quieres para la columna, a la derecha la columna de origen y el cálculo. El resultado sale con los nombres puestos, sin renombrar después.

3.6 Devolver las claves al cuerpo

```
# reset_index devuelve las claves del groupby al cuerpo de la tabla.
resumen = resumen.reset_index()

print(resumen.columns.tolist())
# round sobre una tabla que trae texto adentro avisa, así que se le dice
# exactamente qué columnas redondear.
resumen.sort_values("total", ascending=False).head(5).round(
    {"total": 1, "promedio": 1, "centrales": 1})
```

```
['region', 'tecnologia', 'total', 'promedio', 'centrales']
   region tecnologia      total promedio centrales
5   Biobio      hidro  4151929.7    157.6         3
3   Biobio      carbon  2698912.7    307.3         1
11 Valparaíso      gas  2182672.8    248.5         1
0  Antofagasta      carbon  1967910.4    224.0         1
7   Los Lagos      hidro  1954421.7    111.2         2
```

`reset_index` baja las claves del índice al cuerpo de la tabla, y desde ahí ya se puede ordenar, filtrar o exportar. La combinación más grande es Biobio con hidro.

3.7 size y count

```
# size cuenta filas del grupo. count cuenta valores no nulos de cada columna.
print(gen.groupby("tecnologia").size())
print()
print(gen.groupby("tecnologia")["mwh"].count())
```

```
tecnologia
carbon    17568
diesel     8784
eolica    35136
gas       26352
hidro     43920
solar     43920
dtype: int64

tecnologia
carbon    17568
diesel     8784
eolica    35136
gas       26352
... [y lo mismo para count]
```

`size` cuenta filas del grupo y `count` cuenta valores no nulos de la columna. Acá dan lo mismo porque no faltan datos. Con datos reales, la diferencia entre los dos números es exactamente la cantidad de datos que faltan.

La solución del ejercicio

El ejercicio pedía el promedio y el máximo de demanda por región y día de la semana.

```
resultado = demanda.groupby(["region", "dia_semana"]).agg(
    promedio=("mwh", "mean"),
    maximo=("mwh", "max"),
).reset_index()

print(resultado.shape)
print(resultado.round(1).head(3))
```

El resultado tiene 49 filas, que son 7 regiones por 7 días de la semana. Las tres primeras son Antofagasta los lunes, martes y miércoles, con promedios de 78,1, 78,2 y 78,4 MWh.

Referencia rápida

Instrucción	Para qué sirve
<code>df.groupby("a")["b"].sum()</code>	Agrupar por una clave y sumar
<code>df.groupby(["a", "b"])</code>	Agrupar por varias claves
<code>.agg(["sum", "mean", "std"])</code>	Varias funciones sobre la misma columna
<code>.agg({"a": "sum", "b": "mean"})</code>	Un cálculo distinto por columna
<code>.agg(total=("mwh", "sum"))</code>	Agregación con nombre, la forma recomendada
<code>.reset_index()</code>	Devuelve las claves al cuerpo de la tabla
<code>.size()</code>	Cuenta filas por grupo
<code>.count()</code>	Cuenta valores no nulos por grupo
<code>.nunique()</code>	Cuenta valores distintos
<code>.sort_values("col")</code>	Ordena el resultado

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>KeyError: 'Column not found'</code>	El nombre de la columna dentro de <code>agg</code> está mal escrito.
El resultado tiene menos filas de las esperadas	No es un error. <code>groupby</code> devuelve solo las combinaciones que existen en los datos.
La columna de agrupación desapareció	Quedó en el índice. Se recupera con <code>reset_index</code> .
El promedio no calza con el total dividido por las filas	Hay datos faltantes. Las funciones los saltan, así que el promedio se calculó sobre menos filas. Mira <code>count</code> al lado del promedio.

Tres preguntas de repaso

1. Cuántas filas tiene el resultado de un groupby

Una por grupo existente, no una por dato. Por eso una tabla de 175.680 filas se convierte en una de seis.

2. Cuándo uso sum y cuándo mean

Si el informe pide un total, es sum. Si pide un promedio, es mean. El error clásico es promediar y después multiplicar, que da distinto cuando los grupos tienen distinto número de filas.

3. Qué hace reset_index y por qué se usa tanto

Baja las claves del índice al cuerpo de la tabla. Se usa porque casi siempre uno quiere seguir trabajando con esas claves como columnas normales, o exportarlas.

5. Bloque 4, dar vuelta la tabla

Qué se vio

La libreta y la pizarra. El resultado de un groupby es una tabla larga, una fila por combinación, que es el formato correcto para calcular. Lo que una persona quiere leer es una tabla ancha, con una columna por categoría.

Las dos tienen las mismas cifras. Cambia la forma, no el dato.

LARGO, COMO SALE DE GROUPBY

hora	tecnologia	mwh
0	carbon	265.8
0	solar	0.0
1	carbon	265.7
1	solar	0.0
2	carbon	266.0
2	solar	0.0

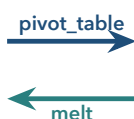
Una fila por combinación. Es el formato que quiere una máquina.

ANCHO, COMO SALE DE PIVOT

hora	carbon	solar
0	265.8	0.0
1	265.7	0.0
2	266.0	0.0

Una columna por tecnología. Es el formato que quiere una persona, y el que se pega en un informe.

Las cifras son las mismas. Cambia la forma, no el dato.



El código, paso a paso

4.1 El perfil horario

```
# index son las filas, columns son las columnas y values lo que se calcula.
perfil = gen.pivot_table(index="hora", columns="tecnologia",
                        values="mwh", aggfunc="mean")

perfil.round(1)
```

```
tecnologia  carbon  diesel  eolica    gas  hidro  solar
hora
0           265.8    0.0    43.4    193.5  139.2    0.0
1           265.7    0.0    45.0    193.1  138.7    0.0
2           266.0    0.0    45.5    193.3  139.2    0.0
3           265.6    0.0    44.2    193.6  139.2    0.0
4           265.7    0.0    44.0    193.2  139.0    0.0
5           265.7    0.0    44.9    193.5  139.1    0.0
6           265.3    0.0    44.2    193.4  138.9    2.7
7           265.8    0.0    44.4    193.7  138.9    9.8
8           265.7    0.0    44.9    192.6  139.0   20.7
9           265.4    0.0    44.8    193.8  138.9   31.6
10          265.4    0.0    44.3    193.2  139.3   40.8
11          265.7    0.0    44.5    192.9  138.9   47.4
... [la tabla sigue hasta la hora 23]
```

index son las filas, columns son las columnas y values es lo que se calcula. Mira la columna de la solar, cero durante la noche y subiendo desde las seis de la mañana. Y mira la del carbón, 265 a las tres de la mañana y 265 al mediodía. Esa diferencia entre una columna plana y una que se prende y se apaga es el problema central de operar un sistema con mucha energía solar.

4.2 El mismo cálculo, en largo

```
# Exactamente el mismo cálculo, presentado como tabla larga.
gen.groupby(["hora", "tecnologia"])["mwh"].mean().round(1).head(8)
```

```
hora  tecnologia
0     carbon      265.8
      diesel       0.0
      eolica      43.4
      gas        193.5
      hidro      139.2
      solar       0.0
1     carbon      265.7
      diesel       0.0
Name: mwh, dtype: float64
```

Las mismas cifras, 265,8 para el carbón a la hora cero. Lo único que cambió es la forma. Acá la hora y la tecnología se repiten en cada fila.

4.3 Huecos y totales

```
# fill_value llena los huecos y margins agrega la fila y la columna de totales.
tabla = gen.pivot_table(index="region", columns="tecnologia", values="mwh",
                        aggfunc="sum", fill_value=0, margins=True)

tabla.round(0)
```

```
tecnologia      carbon  diesel  eolica    gas  hidro  solar \
region
Antofagasta  1967910.0    0.0    0.0    0.0    0.0  479681.0
Atacama       0.0    0.0    0.0    0.0    0.0  191996.0
Biobio       2698913.0    0.0    0.0  1688455.0  4151930.0    0.0
Coquimbo       0.0    0.0  1332263.0    0.0    0.0    0.0
Los Lagos       0.0    0.0    0.0    0.0  1954422.0    0.0
Metropolitana    0.0  40710.0    0.0  1222999.0    0.0    0.0
Valparaiso       0.0    0.0  233637.0  2182673.0    0.0    0.0
All           4666823.0  40710.0  1565899.0  5094126.0  6106351.0  671677.0

tecnologia      All
... [y la fila All con los totales]
```

`fill_value` pone cero donde no hay dato y `margins` agrega la fila y la columna de totales. Cuidado con el cero, porque afirma que hubo cero. Acá significa que esa región no tiene esa tecnología, que es distinto, y en un informe conviene aclararlo con una nota.

4.4 Demanda por región y mes

```
# Demanda promedio por región y mes, con los meses abriéndose hacia el lado.
pd.set_option("display.width", 140)
mensual = demanda.pivot_table(index="region", columns="mes",
                               values="mwh", aggfunc="mean")

mensual.round(0)
```

mes	1	2	3	4	5	6	7	8	9	10	11	12
region												
Antofagasta	63.0	66.0	71.0	78.0	85.0	89.0	90.0	87.0	81.0	75.0	68.0	64.0
Atacama	29.0	30.0	33.0	36.0	39.0	41.0	41.0	40.0	37.0	34.0	31.0	29.0
Biobio	154.0	160.0	173.0	191.0	207.0	216.0	219.0	212.0	198.0	181.0	165.0	155.0
Coquimbo	77.0	80.0	86.0	95.0	103.0	108.0	110.0	106.0	99.0	91.0	83.0	77.0
Los Lagos	81.0	85.0	92.0	101.0	109.0	114.0	116.0	112.0	105.0	96.0	87.0	82.0
Metropolitana	732.0	762.0	824.0	908.0	984.0	1028.0	1044.0	1010.0	942.0	864.0	787.0	739.0
Valparaiso	172.0	179.0	193.0	213.0	231.0	241.0	245.0	237.0	221.0	203.0	184.0	173.0

Esta es la tabla que se pega en un informe. Se ve sola la estacionalidad, la demanda sube en invierno en todas las regiones. Antofagasta pasa de 63 en enero a 90 en julio.

4.5 Deshacer la pivot

```
# melt deshace la pivot. Cada celda de la tabla ancha vuelve a ser una fila.
largo = mensual.reset_index().melt(id_vars="region", var_name="mes",
                                   value_name="mwh_promedio")

print(largo.shape, "que es 7 regiones por 12 meses")
largo.head(4)
```

```
(84, 3) que es 7 regiones por 12 meses
   region mes  mwh_promedio
0  Antofagasta  1      63.251129
1    Atacama  1      28.907567
2    Biobio  1     153.705793
3   Coquimbo  1      76.880538
```

`melt` hace el camino de vuelta, cada celda de la tabla ancha vuelve a ser una fila. Las 84 filas son 7 regiones por 12 meses. Esto es lo que vas a necesitar cuando te devuelvan un Excel con los meses en columnas y tengas que calcular sobre él.

La solución del ejercicio

El ejercicio pedía regiones en las filas, tecnologías en las columnas, suma y totales.

```
mi_pivot = gen.pivot_table(index="region", columns="tecnologia", values="mwh",
                           aggfunc="sum", fill_value=0, margins=True)

mi_pivot.round(0)
```

Es la misma tabla de la celda 4.3, así que puedes compararla para comprobar que quedó bien.

Referencia rápida

Instrucción	Para qué sirve
<code>df.pivot_table(index=, columns=, values=)</code>	Tabla de doble entrada
<code>aggfunc="sum"</code>	El cálculo. Si no se escribe, promedia
<code>fill_value=0</code>	Rellena los huecos con cero
<code>margins=True</code>	Agrega fila y columna de totales
<code>df.melt(id_vars=, var_name=, value_name=)</code>	Deshace la pivot, de ancho a largo
<code>df.to_excel("salida.xlsx")</code>	Exportar la tabla ancha a planilla

Errores comunes

Lo que aparece en pantalla	Qué pasó
Las cifras son más chicas de lo esperado	No escribiste <code>aggfunc</code> y pandas promedió en vez de sumar. Es el error silencioso de este bloque.
<code>ValueError: Index contains duplicate entries, cannot reshape</code>	Usaste <code>pivot</code> en vez de <code>pivot_table</code> y hay más de un valor por celda.
Los meses salen en desorden alfabético	El mes está como texto. Con el mes como número no pasa.
Aparecen NaN en la tabla	Esa combinación no existe en los datos. Decide si corresponde rellenar con cero o dejar el hueco a la vista.

Tres preguntas de repaso

1. Qué hace `pivot_table` si no le dices `aggfunc`

Promedia. Por eso conviene escribirlo siempre, aunque sea para poner `mean`.

2. Qué diferencia hay entre un cero y un hueco en la tabla

El cero afirma que el valor es cero. El hueco dice que no hay dato para esa combinación. En un informe no significan lo mismo.

3. Para qué sirve `melt`

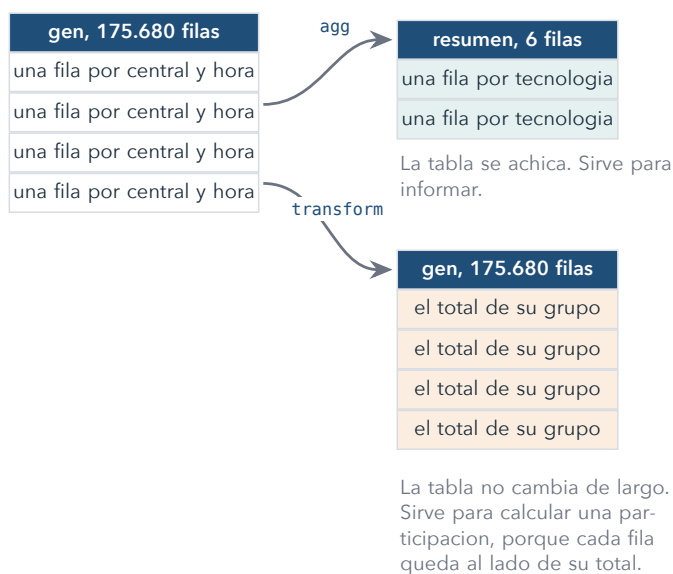
Para volver de la tabla ancha a la larga, que es la forma en que se puede volver a calcular. Es lo que se hace cuando llega un Excel con categorías en columnas.

6. Bloque 5, columnas que no venían en el archivo

Qué se vio

Las etiquetas que pega el patio. La etiqueta de origen, la que trae el archivo, no se toca nunca. Encima se pegan las que uno calcula, y esas son correctas mientras el criterio esté escrito y sea el mismo para todos.

Hay tres tipos. Las que salen de una operación entre columnas, como el factor de planta. Las que salen de una clasificación, como grande o chica. Y las que necesitan saber algo del grupo completo, como la participación de una central dentro de su región.



El código, paso a paso

5.1 Una división entre columnas

```
# Una división entre dos columnas se aplica a las 175.680 filas de una vez.
gen["factor_planta"] = gen["mwh"] / gen["potencia_mw"]

gen[["central", "mwh", "potencia_mw", "factor_planta"]].head(3).round(
    {"mwh": 3, "potencia_mw": 3, "factor_planta": 3})
```

	central	mwh	potencia_mw	factor_planta
0	Central Rio Manso Alto	163.69	420	0.390
1	Central Rio Manso Alto	179.81	420	0.428
2	Central Rio Manso Alto	173.77	420	0.414

Una sola línea aplicada a las 175.680 filas. No hay ciclo ni fórmula que arrastrar. Eso se llama operación vectorizada y es la razón por la que pandas es rápido.

5.2 El factor por tecnología

```
# El factor de planta promedio de cada tecnología, ya con la columna creada.  
gen.groupby("tecnologia")["factor_planta"].mean().round(3).sort_values(ascending=False)
```

```
tecnologia  
gas      0.663  
carbon   0.640  
hidro    0.549  
eolica   0.315  
diesel   0.103  
solar    0.080  
Name: factor_planta, dtype: float64
```

Las que queman combustible arriba, las que dependen del recurso abajo.

OJO CON ESTA CIFRA

El factor de planta de la solar da 0,08 en estos datos y el real chileno ronda 0,19. Los datos del curso son sintéticos y ese número quedó bajo. Lo que sí es real es el orden, el carbón y el gas muy por encima de la solar y la eólica.

5.3 Una clasificación de verdadero y falso

```
# La misma condición del bloque 1, ahora convertida en columna permanente.  
gen["es_renovable"] = gen["tecnologia"].isin(["hidro", "solar", "eolica"])  
  
total = gen.groupby("es_renovable")["mwh"].sum()  
print(total.round(0))  
print("renovable", round(100 * total[True] / total.sum(), 1), "por ciento del total")
```

```
es_renovable  
False    9801659.0  
True     8343928.0  
Name: mwh, dtype: float64  
renovable 46.0 por ciento del total
```

La misma condición del bloque 1, guardada como columna. En estos datos el 46,0 por ciento de la generación es renovable. En el sistema chileno real, contando toda la hidroelectricidad, la cifra de 2025 fue 65,6 por ciento.

5.4 Dos valores según una condición

```
import numpy as np

# np.where recibe la condición, el valor si se cumple y el valor si no.
gen["tamano"] = np.where(gen["potencia_mw"] >= 300, "grande", "chica")

print(gen.groupby("tamano")["central"].nunique())
```

```
tamano
chica      14
grande      6
Name: central, dtype: int64
```

`np.where` es el equivalente de la función SI de la planilla. Recibe la condición, el valor si se cumple y el valor si no. El corte en 300 MW lo decidimos nosotros, no viene del dato, y por eso tiene que quedar escrito.

5.5 Cortar un continuo en categorías

```
# bins son los cortes y labels los nombres. Hay un label menos que cortes.
gen["nivel_uso"] = pd.cut(gen["factor_planta"], bins=[-0.001, 0.3, 0.6, 1.0],
                          labels=["bajo", "medio", "alto"])

print(gen["nivel_uso"].value_counts().sort_index())
```

```
nivel_uso
bajo      66151
medio     49436
alto      60093
Name: count, dtype: int64
```

`pd.cut` corta un número continuo en tramos con nombre. Los cortes son cuatro y los nombres tres, porque los nombres van entre corte y corte, igual que los tramos entre postes de una cerca.

5.6 El total del grupo, fila por fila

```
# transform devuelve el total del grupo fila por fila, en vez de resumirlo.
gen["total_hora_region"] = gen.groupby(
    ["fecha", "hora", "region"])["mwh"].transform("sum")
gen["participacion"] = gen["mwh"] / gen["total_hora_region"]

print(len(gen), "filas antes y después, el largo de la tabla no cambia")
```

```
175680 filas antes y después, el largo de la tabla no cambia
```

Esto es lo nuevo del módulo. `transform` calcula lo mismo que `agg`, pero en vez de devolver una fila por grupo devuelve el valor repetido en cada fila del grupo. Por eso la tabla sigue teniendo 175.680 filas, y por eso se puede dividir una columna por la otra.

5.7 Una hora concreta

```
# Una hora concreta del Biobio, para ver cuánto aportó cada central.
muestra = gen[(gen["fecha"] == "2024-06-15") & (gen["hora"] == 20)
              & (gen["region"] == "Biobio")]

muestra[["central", "mwh", "total_hora_region", "participacion"]].round(
    {"mwh": 3, "total_hora_region": 3, "participacion": 3})
```

	central	mwh	total_hora_region	participacion
4004	Central Rio Manso Alto	296.46	1086.4	0.273
21572	Central Aguas Claras	112.52	1086.4	0.104
39140	Central Tres Saltos	177.35	1086.4	0.163
135764	Termoelectrica Puerto Sur	192.10	1086.4	0.177
153332	Carboelectrica Costa Brava	307.97	1086.4	0.283

Cinco centrales del Biobio a las 20 horas del 15 de junio. El total de la región en esa hora fue 1.086,4 MWh y cada fila trae su participación. Costa Brava aportó el 28,3 por ciento. Las participaciones suman uno, y si no sumaran uno habría un error en las claves.

5.8 Encadenar con assign

```
# assign devuelve una copia con las columnas nuevas, sin tocar el original.
resumen = (gen
            .groupby("tecnologia")
            .agg(total=("mwh", "sum"), centrales=("central", "nunique"))
            .reset_index()
            .assign(promedio_por_central=lambda d: d["total"] / d["centrales"]))

resumen.round(0)
```

	tecnologia	total	centrales	promedio_por_central
0	carbon	4666823.0	2	2333412.0
1	diesel	40710.0	1	40710.0
2	eolica	1565899.0	4	391475.0
3	gas	5094126.0	3	1698042.0
4	hidro	6106351.0	5	1221270.0
5	solar	671677.0	5	134335.0

assign agrega columnas dentro de una cadena, sin cortar el hilo. El lambda recibe la tabla tal como va quedando en ese punto, y por eso puede usar las columnas que se acaban de crear.

5.9 Dejar la tabla presentable

```
# rename cambia nombres, drop elimina columnas y astype convierte tipos.
chica = resumen.rename(columns={"total": "total_mwh"}).drop(columns="centrales")
chica["total_mwh"] = chica["total_mwh"].astype(int)

chica.round(0)
```

	tecnologia	total_mwh	promedio_por_central
0	carbon	4666823	2333412.0
1	diesel	40709	40710.0
2	eolica	1565899	391475.0
3	gas	5094126	1698042.0
4	hidro	6106351	1221270.0
5	solar	671677	134335.0

rename cambia nombres, drop elimina columnas y astype convierte tipos. Son las tres operaciones de terminación, las que se hacen justo antes de entregar.

La solución del ejercicio

El ejercicio pedía una columna de costo estimado y el total por tecnología.

```
COSTO = {"hidro": 5, "solar": 2, "eolica": 3, "gas": 65, "carbon": 45, "diesel": 190}
gen["costo_unitario"] = gen["tecnologia"].map(COSTO)
gen["costo_estimado"] = gen["mwh"] * gen["costo_unitario"]

gen.groupby("tecnologia")["costo_estimado"].sum().round(0).sort_values(ascending=False)
```

El resultado, en dólares, es gas 331.118.195, carbón 210.007.037, hidro 30.531.757, diésel 7.734.807, eólica 4.697.698 y solar 1.343.354. Fíjate en el cambio de orden respecto de la generación. La hidro genera más que el gas y cuesta diez veces menos, y el diésel casi no genera y aun así cuesta más que toda la eólica junta.

Referencia rápida

Instrucción	Para qué sirve
<code>df["nueva"] = df["a"] / df["b"]</code>	Columna calculada entre columnas
<code>df["col"].isin([...])</code>	Columna de verdadero y falso
<code>np.where(cond, si, no)</code>	Dos valores según una condición
<code>np.select([c1, c2], [v1, v2])</code>	Más de dos casos
<code>pd.cut(col, bins=, labels=)</code>	Cortar un continuo en tramos con nombre
<code>.transform("sum")</code>	Sobre un groupby, el total del grupo fila por fila
<code>df["col"].map(diccionario)</code>	Traducir valores con un diccionario
<code>df.assign(nueva=lambda d: ...)</code>	Agregar columnas dentro de una cadena
<code>df.rename(columns={})</code>	Cambiar nombres de columnas
<code>df.drop(columns="a")</code>	Eliminar columnas
<code>df["a"].astype(int)</code>	Convertir el tipo

Errores comunes

Lo que aparece en pantalla	Qué pasó
SettingWithCopyWarning	Estás asignando sobre una tabla que salió de un filtro y pandas no sabe si es copia o vista. Trabaja sobre <code>.copy()</code> o crea la columna sobre la tabla completa.
ValueError: Bin labels must be one fewer than the number of bin edges	En <code>pd.cut</code> pusiste la misma cantidad de nombres que de cortes.
La columna nueva queda llena de NaN	Con <code>map</code> , algún valor de la columna no está en el diccionario.
El resultado de <code>transform</code> cambia el número de filas	No lo cambia. Si lo hizo, usaste <code>agg</code> en vez de <code>transform</code> .

Tres preguntas de repaso

1.Cuál es la diferencia entre `agg` y `transform`

Las dos calculan por grupo. `agg` devuelve una fila por grupo y achica la tabla. `transform` devuelve el resultado repetido en cada fila y deja la tabla del mismo largo.

2. Por qué `pd.cut` lleva un nombre menos que cortes

Porque los nombres van entre corte y corte. Con cuatro marcas se forman tres tramos.

3. Dónde queda guardada una columna nueva

En la tabla que está en memoria. El archivo original no cambia. Para conservarla hay que exportar.

7. Bloque 6, el precio contra la demanda

Qué se vio

La llave de tres dientes. El precio está por fecha, hora y región, y la demanda también. Ninguna de las tres columnas identifica la fila por sí sola, así que la clave del cruce es compuesta y va como lista. Y una segunda lección, que no es de pandas sino de criterio. Una correlación calculada sobre grupos de tamaños muy distintos puede decir algo que no es cierto dentro de ningún grupo.

PRECIOS

fecha	hora	region	precio
01-01	0	Antofagasta	47,97
01-01	1	Antofagasta	40,81

DEMANDA

fecha	hora	region	mwh
01-01	0	Antofagasta	51,95
01-01	1	Antofagasta	49,02

on=["fecha",
"hora",
"region"]

La fecha sola no identifica la fila, y la región sola tampoco. La fila queda identificada por las tres columnas juntas. Si se une por una sola de ellas, cada fila encuentra muchas parejas y el resultado se infla.

5.208 filas de precios con 5.208 de demanda dan 5.208 filas unidas. Ese es el número que hay que mirar después de unir.

El código, paso a paso

6.1 La respuesta de la API, otra vez

```
import json

# La respuesta de la API del Lab 01, ahora como punto de partida.
with open("precios_nudo.json") as f:
    cuerpo = json.load(f)
    precios = pd.json_normalize(cuerpo["datos"])
    precios["fecha"] = pd.to_datetime(precios["fecha"])

print(precios.shape)
precios.head(3)
```

```
(5208, 4)
   fecha  hora  region  precio_usd_mwh
0 2024-01-01    0  Antofagasta         47.97
1 2024-01-01    1  Antofagasta         40.81
2 2024-01-01    2  Antofagasta         49.35
```

Es el mismo JSON del Módulo 1, los precios de nudo de enero. La línea de `pd.to_datetime` es obligatoria, porque la fecha llega como texto desde el JSON y sin convertirla el cruce no calza.

6.2 El cruce por tres columnas

```
# Con una lista en on, el merge cruza por las tres columnas juntas.
enero = demanda[demanda["mes"] == 1]
mix = pd.merge(precios, enero, on=["fecha", "hora", "region"], how="inner")

print(mix.shape[0], "filas, que son las horas de enero por las siete regiones")
mix[["fecha", "hora", "region", "precio_usd_mwh", "mwh"]].head(3)
```

```
5208 filas, que son las horas de enero por las siete regiones
   fecha hora region precio_usd_mwh  mwh
0 2024-01-01    0  Antofagasta      47.97  51.95
1 2024-01-01    1  Antofagasta      40.81  49.02
2 2024-01-01    2  Antofagasta      49.35  48.22
```

Con una lista en on, el merge cruza por las tres columnas juntas. 5.208 filas de precios, 5.208 de demanda de enero, 5.208 unidades. Ese es el número que hay que mirar. Si hubieras unido solo por fecha, el resultado tendría más de ochocientas mil filas y ninguna advertencia habría salido.

6.3 La correlación global

```
# Correlación entre precio y demanda sobre las 5.208 filas juntas.
# float() saca el envoltorio de numpy y deja el número pelado.
print(round(float(mix["precio_usd_mwh"].corr(mix["mwh"])), 3))
```

```
0.979
```

0,979 es una relación casi perfecta. La correlación mide si dos cosas se mueven juntas, entre menos uno y uno, y no dice que una cause la otra.

6.4 La misma correlación, por región

```
# La misma correlación, calculada dentro de cada región por separado.
mix.groupby("region").apply(
    lambda x: x["precio_usd_mwh"].corr(x["mwh"]), include_groups=False
).round(3)
```

```
region
Antofagasta    0.199
Atacama        0.125
Biobio         0.455
Coquimbo       0.203
Los Lagos      0.300
Metropolitana  0.917
Valparaiso     0.517
dtype: float64
```

Dentro de cada región la relación es mucho más débil, entre 0,125 en Atacama y 0,917 en la Metropolitana. La correlación global estaba midiendo, en parte, que las regiones grandes tienen a la vez más demanda y precios más altos. Las dos cifras son correctas, y la pregunta es cuál responde lo que se preguntó.

La solución del ejercicio

El ejercicio pedía el precio promedio por hora en la Metropolitana.

```
seleccion = mix[mix["region"] == "Metropolitana"]

print(seleccion.groupby("hora")["precio_usd_mwh"].mean().round(2))
```

El máximo queda en la hora 19, con 145,11 dólares por MWh, y el mínimo en la hora 3, con 101,68.

OJO CON ESTA FORMA

En estos datos la punta está en la tarde, cerca de las 19 horas. En el sistema eléctrico chileno real los máximos mensuales de demanda de 2025 cayeron todos entre las 11 y las 16 horas, y todos en día hábil. La forma de la curva del laboratorio no reproduce la del sistema real, y conviene saberlo antes de citar una hora de punta en un informe.

Referencia rápida

Instrucción	Para qué sirve
<code>pd.merge(a, b, on=["f", "h", "r"])</code>	Cruzar por una clave compuesta
<code>left_on, right_on</code>	Cuando las columnas se llaman distinto
<code>pd.to_datetime(col)</code>	Convertir a fecha antes de cruzar
<code>a["x"].corr(a["y"])</code>	Correlación entre dos columnas
<code>method="spearman"</code>	Correlación por rangos, más robusta con saltos
<code>df.groupby("g").apply(...)</code>	Repetir un cálculo dentro de cada grupo

Errores comunes

Lo que aparece en pantalla	Qué pasó
El cruce devuelve muchas más filas de las esperadas	La clave está incompleta. Con una sola columna cada fila encuentra muchas parejas.
El cruce devuelve cero filas	Los tipos no calzan. Lo más común es que una fecha sea texto y la otra sea fecha.
<code>ValueError: You are trying to merge on object and datetime64 columns</code>	Es el caso anterior, dicho por pandas.

Tres preguntas de repaso

1. Por qué la clave de este cruce necesita tres columnas

Porque hay 24 horas para cada fecha y 7 regiones para cada hora. Solo las tres juntas identifican una fila.

2. Qué comprobación se hace después de cruzar

Contar las filas y compararlas con lo que se esperaba. Acá 5.208 antes y 5.208 después.

3. Por qué la correlación global es más alta que la de cada región

Porque mezcla regiones de tamaños muy distintos, y esa diferencia entre regiones se suma a la relación que hay dentro de cada una.

8. Bloque 7, la misma película en Polars

Qué se vio

La grúa nueva otra vez, ahora para las maniobras de siempre. Filtrar, agrupar y crear columnas existen igual en Polars. Cambia la escritura y no cambia la idea.

La diferencia visible es que Polars no repite el nombre de la tabla dentro de la condición, usa `pl.col` para referirse a la columna, y construye el resultado con expresiones.

El código, paso a paso

7.1 La misma tabla, en Polars

```
import polars as pl

# El mismo archivo y la misma unión del principio, escritos en Polars.
gen_pl = pl.read_csv("generacion.csv", try_parse_dates=True).join(
    pl.read_csv("centrales.csv"), on="central", how="left")

print(gen_pl.shape)
```

```
(175680, 8)
```

El mismo archivo y la misma unión del principio, con `join` en vez de `merge`. Las mismas 175.680 filas.

7.2 Filtrar

```
# pandas. Corchetes, y el nombre de la tabla repetido en cada condición.
print("pandas", gen[(gen["tecnologia"] == "solar")
                    & (gen["region"] == "Antofagasta")].shape[0], "filas")

# Polars. filter con expresiones pl.col, sin repetir el nombre de la tabla.
print("polars", gen_pl.filter((pl.col("tecnologia") == "solar")
                             & (pl.col("region") == "Antofagasta")).shape[0], "filas")
```

```
pandas 26352 filas
polars 26352 filas
```

Arriba pandas, abajo Polars, y el mismo resultado de 26.352 filas.

7.3 Agrupar y resumir

```
# pandas. agg con tuplas de columna y función.
print(gen.groupby("tecnologia").agg(total=("mwh", "sum"),
                                   promedio=("mwh", "mean")).round(1))
```

	total	promedio
tecnologia		
carbon	4666823.0	265.6
diesel	40709.5	4.6
eolica	1565899.4	44.6
gas	5094126.1	193.3
hidro	6106351.4	139.0
solar	671677.2	15.3

```
# Polars. group_by con expresiones, y el nombre a la izquierda igual que antes.
gen_pl.group_by("tecnologia").agg(
    total=pl.col("mwh").sum(), promedio=pl.col("mwh").mean()).sort("total")
```

```
shape: (6, 3)
+-----+-----+-----+
| tecnologia | total | promedio |
| --- | --- | --- |
| str | f64 | f64 |
+=====+=====+=====+
| diesel | 40709.51 | 4.634507 |
| solar | 671677.15 | 15.293196 |
| eolica | 1.5659e6 | 44.566808 |
| carbon | 4.6668e6 | 265.643388 |
| gas | 5.0941e6 | 193.310795 |
| hidro | 6.1064e6 | 139.033502 |
+-----+-----+-----+
```

La agregación con nombre se escribe casi igual. Las cifras son idénticas. El orden de las filas no lo es, porque Polars no garantiza el orden de una agrupación si uno no lo pide, y por eso se agregó sort.

7.4 Crear una columna

```
# pandas crea la columna asignando a un nombre que no existe.
print(gen.assign(factor=gen["mwh"] / gen["potencia_mw"])["factor"].mean().round(3))
```

```
# Polars usa with_columns, y alias para ponerle nombre al resultado.
print(gen_pl.with_columns(
    (pl.col("mwh") / pl.col("potencia_mw")).alias("factor"))["factor"].mean())
```

```
0.389
0.3888563053764884
```

En pandas se asigna, en Polars se usa with_columns con un alias. El promedio del factor de planta da lo mismo en las dos, 0,389.

Referencia rápida

pandas	Polars
<code>df[df["a"] == 1]</code>	<code>df.filter(pl.col("a") == 1)</code>
<code>df.groupby("a").agg(...)</code>	<code>df.group_by("a").agg(...)</code>
<code>df["nueva"] = ...</code>	<code>df.with_columns(...).alias("nueva")</code>
<code>pd.merge(a, b, on=)</code>	<code>a.join(b, on=)</code>
<code>df.sort_values("a")</code>	<code>df.sort("a")</code>
<code>df.to_pandas()</code>	Convierte de Polars a pandas

Tres preguntas de repaso

1. Cuál conviene aprender

pandas, que es lo que usa casi todo el mundo. Polars se aprende después, cuando aparezca un archivo que lo justifique.

2. Se pueden mezclar las dos

Sí, y es lo habitual. Se lee y se filtra con Polars y, cuando el resultado ya es chico, se pasa a pandas con `to_pandas`.

3. Por qué el resultado sale en otro orden

Porque Polars no garantiza el orden de una agrupación. Se pide con `sort`, y no asumir el orden es buena costumbre en las dos herramientas.

Hoja de referencia del Módulo 2

Para imprimir y tener al lado

ELEGIR

<code>df[["a", "b"]]</code>	Columnas por nombre
<code>df.loc[0:4, ["a"]]</code>	Por nombre, incluye las dos puntas
<code>df.iloc[0:4, [2]]</code>	Por posición, deja fuera la última
<code>df[(df["a"] > 1) & (df["b"] == "x")]</code>	Filas, cada condición entre paréntesis
<code>df["a"].isin([...])</code>	El valor está en la lista
<code>df["a"].between(a, b)</code>	El valor está dentro del rango

FECHAS

<code>df["f"].dt.month, .dt.quarter</code>	Mes y trimestre
<code>df["f"].dt.dayofweek</code>	Día de la semana, 0 es lunes
<code>df["f"].dt.day_name()</code>	Nombre del día
<code>df["f"].between("2024-06-01", "2024-08-31")</code>	Rango de fechas

AGRUPAR Y RESUMIR, LO QUE MÁS SE USA

<code>df.groupby("a")["b"].sum()</code>	Una clave, un cálculo
<code>df.groupby(["a", "b"]).agg(</code>	Varias claves y agregación con nombre
<code>total=("mwh", "sum"),</code>	El nombre a la izquierda
<code>promedio=("mwh", "mean"))</code>	La columna y el cálculo a la derecha
<code>.reset_index()</code>	Devuelve las claves al cuerpo de la tabla

DAR VUELTA Y CREAR COLUMNAS

<code>df.pivot_table(index=, columns=, values=, aggfunc=)</code>	Tabla de doble entrada, y si no se dice el cálculo, promedia
<code>df.melt(id_vars=)</code>	De ancho a largo
<code>df["n"] = df["a"] / df["b"]</code>	Columna calculada
<code>np.where(cond, si, no)</code>	Dos valores según una condición
<code>pd.cut(col, bins=, labels=)</code>	Tramos con nombre, un nombre menos que cortes
<code>.transform("sum")</code>	El total del grupo, fila por fila

CRUZAR

<code>pd.merge(a, b, on=["fecha", "hora", "region"])</code>	Clave compuesta, va como lista
<code>len(resultado)</code>	Después de cruzar, siempre

Las cinco cosas que hay que recordar del módulo

- Ninguna de estas operaciones cambia la tabla original, todas producen una tabla nueva.
- Filtrar filas es pasar una columna de verdaderos y falsos, y cada condición va entre paréntesis.
- `groupby` devuelve una fila por grupo, y con `agg` hace varios cálculos de una vez.
- Largo y ancho son la misma cifra con dos formas, una para calcular y otra para leer.
- Cuando un resultado sorprenda, cuenta las filas. La mitad de los errores se detectan así.